

**АНАЛИТИКА И ПРЕДСКАЗАНИЕ
ФИНАНСОВЫХ ВРЕМЕННЫХ РЯДОВ**

Мещеряков Н. С.

Преподаватель:
Киселев С. Д.

Содержание

Введение	3
Описание задачи	4
Стек технологий	5
Свойства финансовых временных рядов	6
Методы прогнозирования	8
Сравнение результатов	14
Заключение и выводы	20
Литература	23

Введение

Под понятием временных рядов (time series) понимаются массивы данных, содержащие наблюдения за какими-либо переменными в разные моменты времени. Например, ежедневные данные о количестве клиентов, посетивших кафе, или же ежечасное измерение температуры в городе на протяжении дня.

Временные ряды играют огромную роль в эконометрике. Современные финансовые рынки представляют собой сложные динамические системы, генерирующие огромные объемы данных. Анализ финансовых временных рядов играет ключевую роль в принятии инвестиционных решений, управлении рисками, прогнозировании рыночной динамики и разработке автоматизированных торговых стратегий. В отличие от традиционных статистических данных, финансовые временные ряды обладают рядом особенностей, таких как нестационарность, наличие шумов, аномальных выбросов, что требует применения специализированных методов анализа.

Актуальность исследования финансовых временных рядов обусловлена растущим интересом к алгоритмической торговле, машинному обучению в финансах и риск-менеджменту. Традиционные подходы, такие как ARIMA, SSA (сингулярный спектральный анализ) и методы скользящего среднего, дополняются современными технологиями, включая глубокое обучение (LSTM, Transformer) и ансамблевые методы. Однако, несмотря на развитие вычислительных методов, остаются открытыми вопросы, связанные с интерпретируемостью моделей, устойчивостью к рыночным шокам и адаптацией к быстро меняющимся условиям.

Описание задачи

В данной работе основной задачей является разработка эффективных методов прогнозирования временных рядов, позволяющих минимизировать ошибки предсказаний и адаптироваться к изменяющимся рыночным условиям.

Основные задачи работы:

1. Анализ существующих методов прогнозирования - от статистических (ARIMA, SSA) до современных подходов машинного обучения (LSTM, градиентный бустинг) и их гибридных комбинаций.
2. Экспериментальное сравнение моделей, где проводится комплексная оценка их работы по трем ключевым направлениям: измерение точности прогнозирования с использованием общепринятых метрик MAE, MSE и MAPE, проверка устойчивости к характерным рыночным шумам и выбросам, а также анализ вычислительной эффективности с точки зрения скорости обучения и выполнения прогнозов.
3. Разработка программного решения, предусматривающая создание удобного инструмента для аналитиков с возможностями проведения анализа конкретного финансового временного ряда (курсы акций, валют), гибкого выбора и настройки прогнозных моделей и выполнения прогнозирования на заданный период.
4. Практическая валидация полученных результатов, включающая тестирование на реальных рыночных данных различных классов активов, сравнение эффективности с базовыми методами прогнозирования и оценку практической применимости разработанных решений в реальных торговых стратегиях.

Стек технологий

Данная работа выполнена на языке программирования Python 3 с использованием следующих сторонних библиотек:

- Работа с данными табличного вида: Numpy, Pandas
- Визуализация временных рядов: Seaborn, Matplotlib
- Классические инструменты машинного обучения: scikit-learn, lightgbm
- Библиотеки для проектирования нейросетей и работы с ними: Tensorflow, PyTorch
- Статистические модели: statsmodels, pmdarima, NeuralProphet
- Запросы к API: requests

Проект выполнен в среде разработки "Google Colab"

Свойства финансовых временных рядов

Финансовые временные ряды обладают рядом характерных свойств, которые отличают их от других типов данных и которые необходимо учитывать при построении прогнозных моделей. Во-первых, они демонстрируют нестационарность, то есть их статистические свойства (среднее, дисперсия) изменяются во времени. Это проявляется в наличии трендов (долгосрочных направленных движений). Во-вторых, часто наблюдается сезонность — повторяющиеся паттерны, связанные с временем года, дня недели или времени суток (для внутридневных данных).

Еще одной важной особенностью является автокорреляция — зависимость текущих значений от предыдущих, особенно выраженная для абсолютных значений доходностей. Финансовые ряды также демонстрируют "тяжелые хвосты" распределения, когда экстремальные события происходят чаще, чем предполагает нормальное распределение. Кроме того, для них характерен эффект рычага — негативные шоки обычно вызывают большую волатильность, чем позитивные шоки той же величины. Эти свойства требуют применения специальных методов анализа, таких как модели ARIMA, GARCH или машинное обучение с учетом временной структуры данных. Понимание этих особенностей критически важно для построения адекватных прогнозных моделей и эффективного управления финансовыми рисками.

Важно понимать, что все эти факторы делают качественное прогнозирование не всегда осуществимым. Очень многое зависит от свойств конкретного временного ряда. Например, поведение курсов валют отличается от изменений биржевых акций, а данные, взятые за конкретный промежуток времени, могут стремительно возрастать или убывать из-за мировых событий.

Учитывая данные особенности, будет логично создать программный комплекс, предоставляющий возможность выбора временного ряда для анализа и прогнозирования. Пользователь сможет явно указывать, к какой валюте или акции он хочет получить прогноз, а также какое количество данных за последние дни будет использовано для обучения моделей и анализа трендов.

Методы прогнозирования

Начнем с загрузки и предобработки данных. Данные будут браться из API запросов url "https://api.twelvedata.com/time_series". Этот сервис обладает возможностью предоставлять данные за последние n промежутков, при этом можно настроить запрос на конкретную валюту и количество дней. В функции preprocess_data я прописал код приведения временного ряда к конкретному типу, удобному для работы, а также заполнил пропуски в данных при помощи интерполяции (наиболее предпочтительный способ восполнения).

```
def preprocess_data(data, interval, data_size):  
  
    df = pd.DataFrame(data["values"])  
  
    if 'datetime' in df.columns:  
        df['datetime'] = pd.to_datetime(df['datetime'])  
        df.set_index('datetime', inplace=True)  
  
    df.drop(columns=["high", "low", "open", "volume"], inplace=True, errors='ignore')  
  
    freq_map = {  
        "1day": "D",  
        "1hour": "H",  
        "5min": "5T"  
    }  
  
    full_index = pd.date_range(  
        start=df.index.min(),  
        end=df.index.max(),  
        freq=freq_map[interval]  
    )  
  
    df['close'] = df['close'].interpolate(method='time').ffill().bfill()  
  
    return df
```

Теперь можно переходить к апробации методов прогнозирования. Для начала воспользуемся "классикой" временных рядов – моделями вида ARIMA. В качестве пайплайна будем использовать ARIMA с подбором гиперпараметров (как в auto_arima). Важно понимать, что для использования таких моделей необходимо изучить временной ряд, исследовать его на стационарность. Для этого воспользуемся тестом Дики-Фуллера.


```
def check_stationarity(timeseries):  
    if timeseries.isna().any() or np.isinf(timeseries).any():  
        print("Обнаружены пропуски в данных")  
        timeseries = timeseries.replace([np.inf, -np.inf], np.nan)  
        timeseries = timeseries.dropna()  
  
    if len(timeseries) < 2:  
        #мало данных для проверки  
        return False  
  
    try:  
        timeseries_array = timeseries.values  
        result = adfuller(timeseries_array)  
        print('Тест Дики-Фуллера:')  
        print(f'ADF статистика: {result[0]}')  
        print(f'p-значение: {result[1]}')  
        print('Критические значения:')  
        for key, value in result[4].items():  
            print(f'\t{key}: {value}')  
  
        #выбираем стандартное пороговое значение  
        return result[1] < 0.05  
  
    except Exception as e:  
        print(f"Ошибка при проверке стационарности: {str(e)}")  
        return False
```

В случае, если проверка показала, что ряд не стационарен, будем брать разности первого порядка и делать проверку снова. Теперь можно запустить перебор параметров ARIMA. Выбор лучшей модели будет производиться на основании критерия AIC, он позволит найти баланс между точностью и сложностью модели.

```
def find_best_arma_params(data, max_p=5, max_d=2, max_q=5):
    best_aic = float('inf')
    best_order = None

    total_combinations = (max_p + 1) * (max_d + 1) * (max_q + 1)
    current_combination = 0

    for p in range(max_p + 1):
        for d in range(max_d + 1):
            for q in range(max_q + 1):
                current_combination += 1
                print(f"\rПорядок: {current_combination}/{total_combinations} комбинаций", end="")

                try:
                    model = ARIMA(data, order=(p, d, q))
                    results = model.fit()
                    if results.aic < best_aic:
                        best_aic = results.aic
                        best_order = (p, d, q)
                    print(f"\nлучшие параметры: {best_order} (AIC: {best_aic:.2f})")
                except:
                    continue

    print(f"\n\nЛучшие параметры ARIMA: {best_order}")
    return best_order
```

После этого делаем предсказание и считаем итоговые метрики на валидации. Анализ качества проведем позже, после использования других моделей.

Далее воспользуемся моделью NeuralProphet. Он является улучшенной версией классического Prophet (позволяет сочетать статистические методы с нейросетями). Свойства данного подхода позволят нам проследить более сложные зависимости в данных. Явно укажем количество лагов (7) при настройке модели, а также тип трендов (линейный), learning rate и количество эпох обучения. Снова проведем оценку, замерим все необходимые метрики.

```
model = NeuralProphet(  
    growth="linear",  
    changepoints=None,  
    n_changepoints=5,  
    changepoints_range=0.8,  
    trend_reg=0.05,  
    trend_reg_threshold=False,  
    yearly_seasonality="auto",  
    weekly_seasonality="auto",  
    daily_seasonality="auto",  
    seasonality_mode="multiplicative",  
    seasonality_reg=0.1,  
    n_forecasts=len(test_data),  
    n_lags=7,  
    learning_rate=0.001,  
    epochs=50,  
    batch_size=64,  
    loss_func="MSE",  
    optimizer="AdamW"  
)
```

Перейдем к нейросетям. В данном исследовании была использована LSTM-модель как наиболее привычный тип нейросетей для работы с последовательностями. В написанной мной функции пользователь может сам настраивать количество LSTM-слоев в модели, выбирать функцию активации.

```
def create_lstm_model(seq_length, n_features):  
    model = Sequential([  
        LSTM(50, activation='relu', input_shape=(seq_length, n_features), return_sequences=True),  
        Dropout(0.2),  
        LSTM(50, activation='relu'),  
        Dropout(0.2),  
        Dense(1)  
    ])  
    model.compile(optimizer='adam', loss='mse')  
    return model
```

Также добавим дополнительные фичи в наши данные (лаги, скользящее среднее и тд.)

```
def add_technical_indicators(df):
    df['SMA_20'] = df['close'].rolling(window=20).mean()
    df['SMA_50'] = df['close'].rolling(window=50).mean()

    delta = df['close'].diff()
    gain = (delta.where(delta > 0, 0)).rolling(window=14).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=14).mean()
    rs = gain / loss
    df['RSI'] = 100 - (100 / (1 + rs))

    exp1 = df['close'].ewm(span=12, adjust=False).mean()
    exp2 = df['close'].ewm(span=26, adjust=False).mean()
    df['MACD'] = exp1 - exp2
    df['Signal_Line'] = df['MACD'].ewm(span=9, adjust=False).mean()

    return df
```

Теперь воспользуемся так называемым "королем машинного обучения" – градиентным бустингом. Была выбрана библиотека lightgbm из-за быстрого обучения и легкости настройки.

```
# Создание и обучение модели
model = lgb.LGBMRegressor(n_estimators=200, learning_rate=0.05)
model.fit(X_train, y_train)
```

И, наконец, опробуем метод сингулярного спектрального анализа (SSA). Напишем функции, выполняющие спектральное разложение матрицы, а затем ее восстановление в исходный временной ряд. В качестве метода прогнозирования следующих значений будем использовать авторегрессионный подход.

```
def ssa_decompose(ts, window):
    N = len(ts)
    K = N - window + 1
    X = np.column_stack([ts[i:i+K] for i in range(window)])
    U, s, Vt = svd(X, full_matrices=False)
    return U, s, Vt

def ssa_reconstruct(U, s, Vt, r):
    X_elem = np.zeros((U.shape[0], Vt.shape[1]))
    for i in range(r):
        X_elem += s[i] * np.outer(U[:, i], Vt[i, :])
    L, K = X_elem.shape
    N = L + K - 1
    recon = np.zeros(N)
    counts = np.zeros(N)
    for i in range(L):
        for j in range(K):
            recon[i + j] += X_elem[i, j]
            counts[i + j] += 1
    return recon / counts
```

```
def ssa_forecast(ts, window, r, n_forecast):
    U, s, Vt = ssa_decompose(ts, window)
    r = min(20, window)
    recon = ssa_reconstruct(U, s, Vt, r)
    p = window
    X = np.array([recon[i:i+p] for i in range(len(recon) - p)])
    y = recon[p:]
    coef, _, _, _ = lstsq(X, y)
    forecast = []
    last_vals = list(recon[-p:])
    for _ in range(n_forecast):
        next_val = np.dot(coef, last_vals[-p:])
        forecast.append(next_val)
        last_vals.append(next_val)
    return forecast
```

Сравнение результатов

Для наиболее объективного сравнения будем проводить 3 замера метрик для каждой модели на одинаковых наборах данных, а затем усреднять полученные значения (для MAPE). MSE и MAE для каждой модели будут представлены для временного ряда номер 3. В качестве обучающих и валидационных выборок были выбраны более отличающиеся по структуре и изменчивости временные ряды, а именно:

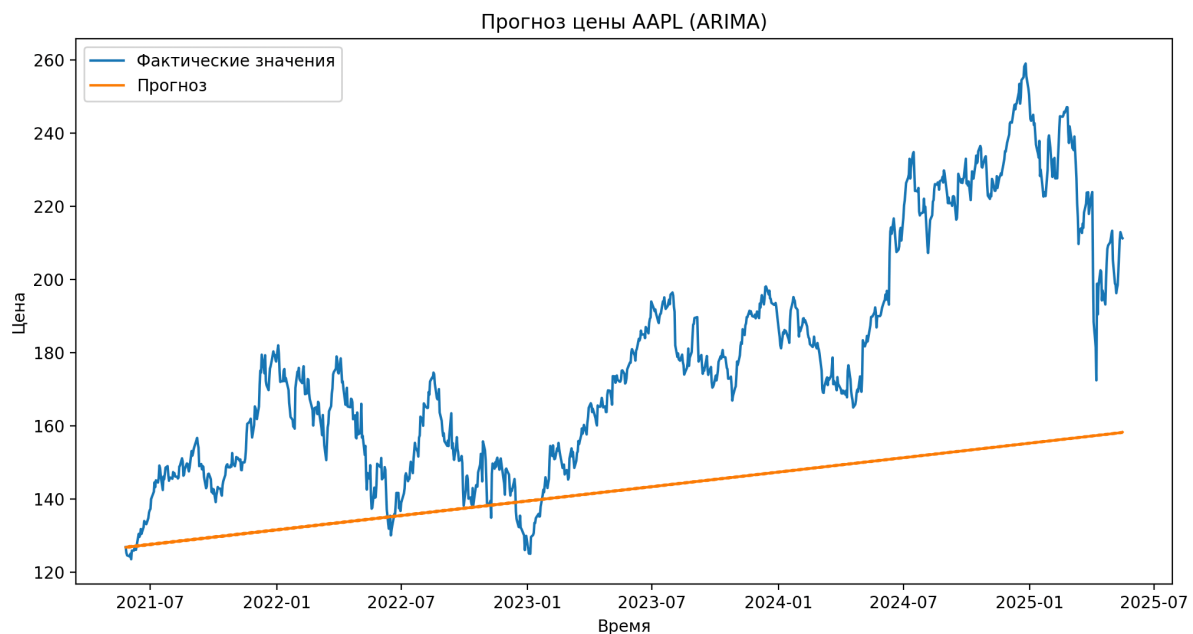
1. USD/RUB с периодичностью 1 час
2. USD/EUR с периодичностью 1 день
3. AAPL (акции Apple) с периодичностью 1 день

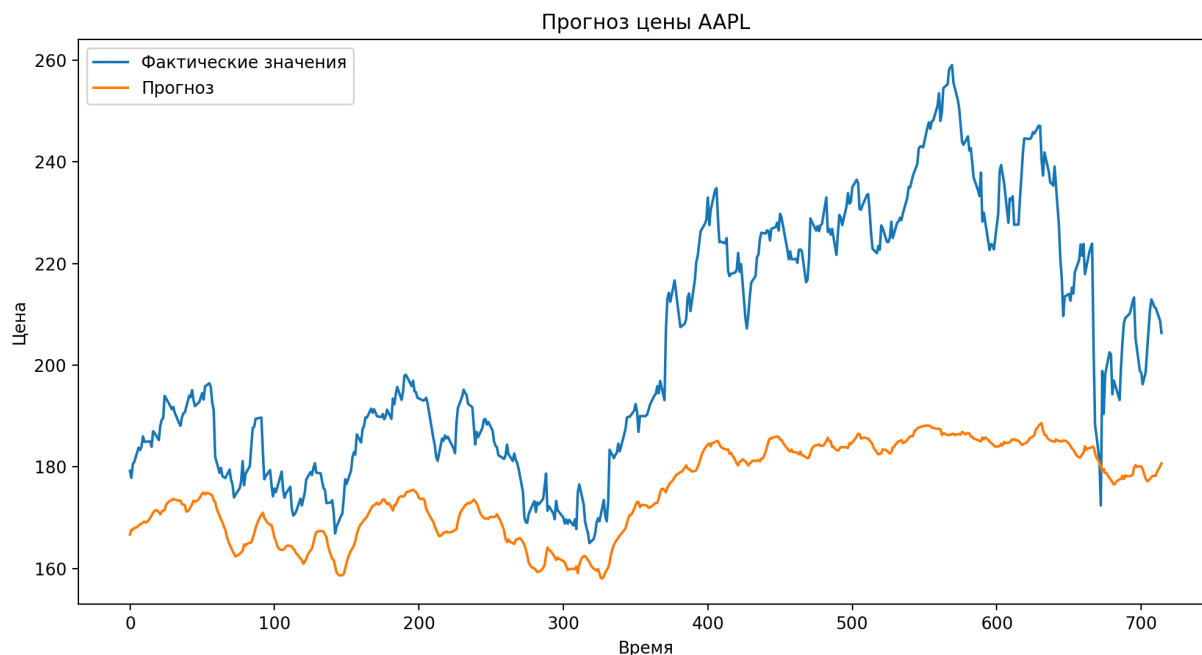
Все наборы данных будут иметь одинаковый размер (5000 последних значений). 90% значений (4500 значений) будут использованы для обучения моделей, а на оставшихся 10% (500 значений) будет проведена валидация.

В итоге получаем такие результаты:

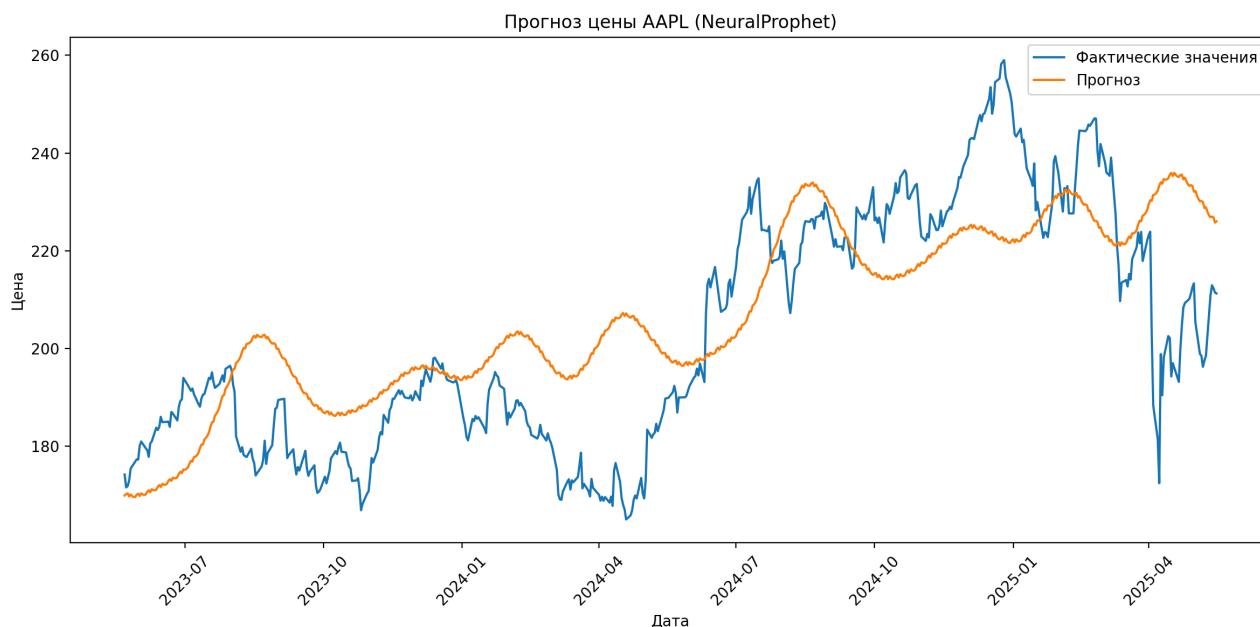
Метрика	NeuralProphet	LSTM-модель	ARIMA	LightGBM	SSA
MSE (в ед.)	1003.74	1051.23	1415.41	1275.62	952.55
MAE (в ед.)	27.36	28.13	29.36	27.29	24.84
MAPE (в %)	6,43	8,56	9,3	8,8	7,31

Большая часть "неудачных прогнозов" пришлась на 3-ий временной ряд, в основном ответы моделей лежат "ниже" или "выше" настоящих значений. Это можно объяснить нестабильностью акций Apple, где явно не прослеживается тренд в данных. Например, ARIMA и SSA смогли спрогнозировать только линейный возрастающий тренд (прогноз SSA близок к линейности). Тем не менее, общую тенденцию возрастания данных модели смогли предсказать.



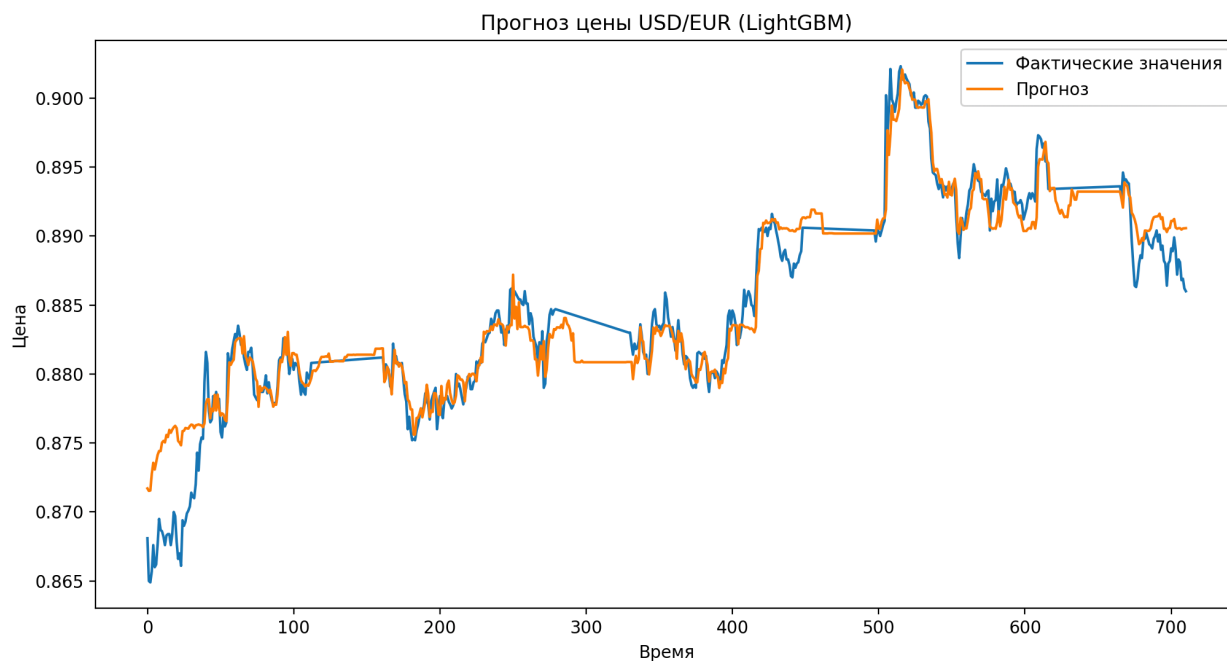


Здесь лучше остальных моделей проявил себя NeuralProphet. Несмотря на то, что его предсказания все равно оказались далеки от истины, он смог распознать и линейный возрастающий тренд в данных, и изменения с периодичностью приблизительно 6 месяцев – год.

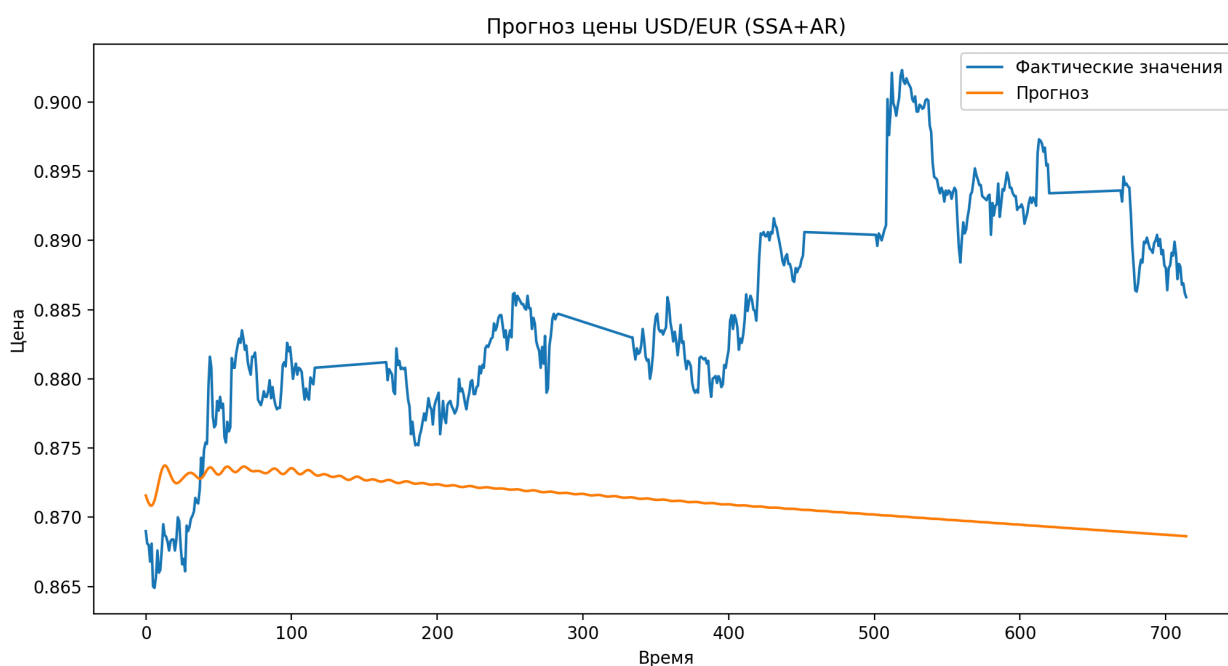


Лучше всего модели справились со 2-м временным рядом (USD/EUR), который содержал более простые для анализа данные. Почти все представленные методы смогли проследить тенденции и подстроить под них предсказания.

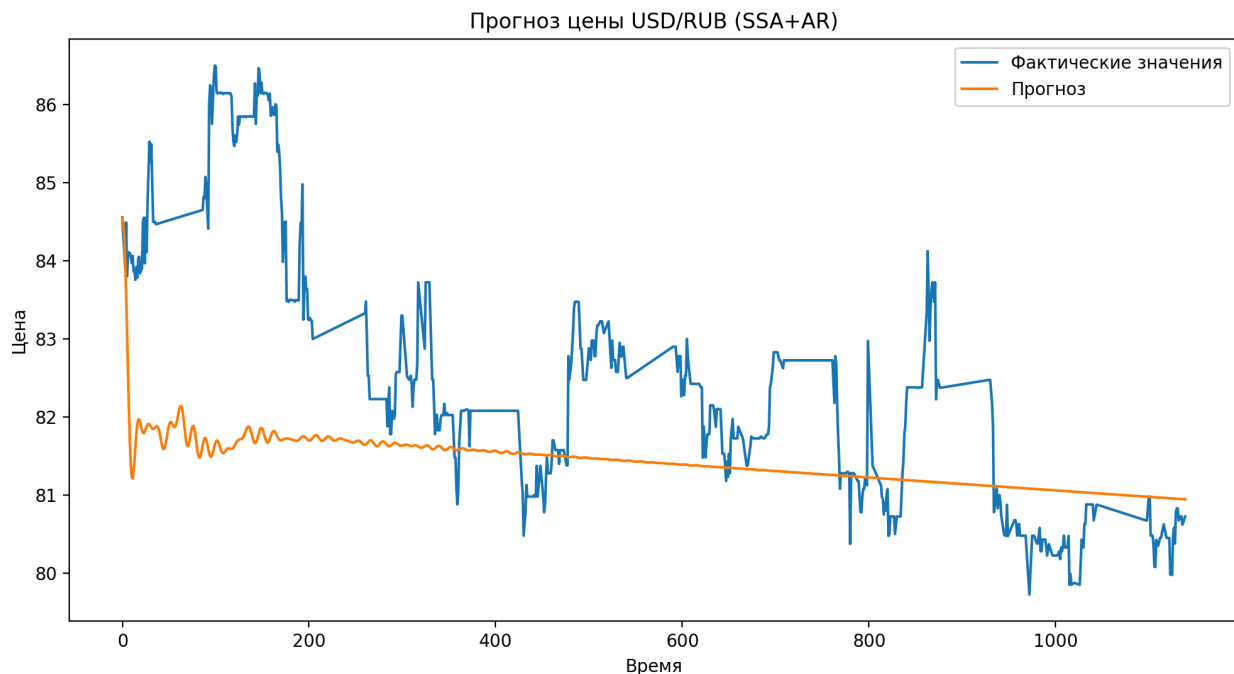
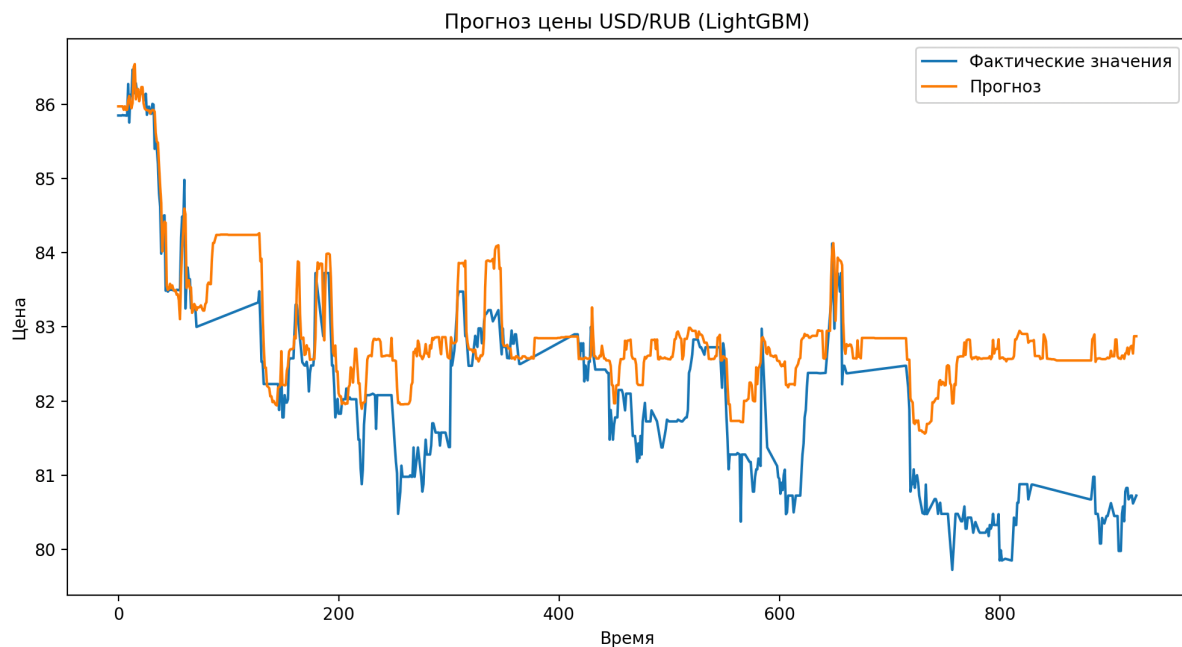
Отдельно стоит отметить LightGBM, он показал наилучший результат по метрикам (что можно увидеть по графику).



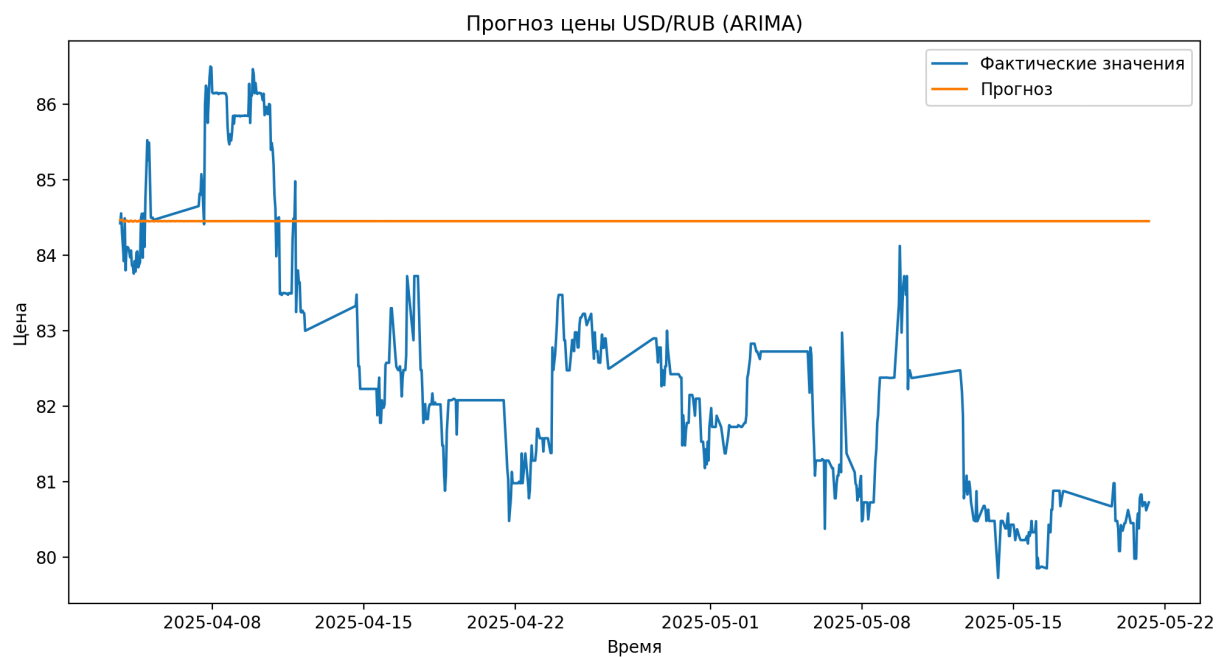
Однако авторегрессионный подход к прогнозированию с помощью SSA снова не смог показать достойного качества.



Третий набор данных содержал множество "скачков" стоимости, поэтому качественного прогноза получить не удалось. Однако глобальное падение цены в начале прогнозируемого промежутка модели все же проследить смогли.



ARIMA, к сожалению, показала себя не так хорошо на этих данных.



Заключение и выводы

Сравнивая качество использованных методов, можно сделать выводы о релевантности использования данных подходов для прогнозирования финансовых временных рядов.

Не очень хорошо на практике себя показал сингулярный спектральный анализ с авторегрессией (SSA+AR), в большей части случаев он предсказывает значения линейным трендом. Хотя в ходе исследований действительно были получены весьма высокие метрики, по графикам можно убедиться, что данный метод почти не выявляет скрытых трендов в данных. Основная идея его использования была взята из журнала Высшей Школы Экономики, ссылка на материал: <https://bijournal.hse.ru/data/2023/10/02/2062423607/6.pdf>. Возможно, автор статьи в своих тестированиях использовал не авторегрессионный подход для прогнозирования, а получал предсказания значений акций другим методом, что в материалах исследования отмечено не было. Например, подавал разложение ряда на вход другой модели.

ARIMA с подбором гиперпараметров тоже не смогла положительно проявить себя в решении поставленной задачи. Модель редко находит статистические тренды в данных и подстраивается под них, ограничиваясь линейным предсказанием. Скорее всего, основной проблемой является то, что даже после многократного дифференцирования ряды остаются нестационарными. Возможно, стоило увеличить перебор параметра d для модели, но, вероятно, проблема находится в самих данных, а точнее в их нестабильности.

Хорошо себя зарекомендовали в данной задаче методы машинного обучения – LightGBM и LSTM-модель. В метриках модели показали схожие результаты, что можно обусловить отсутствием статистической составляющей и одинаковым набором признаков (специально для этих моделей был добавлен подсчет лагов, скользящих средних и тд.) Эти методы умеют отслеживать сложные зависимости в данных и подстраивать свои предсказания под них, несмотря на отсутствие статистического анализа. Это в очередной раз доказывает, что градиентный бустинг – один из лучших ML ансамблей, способный качественно решать самые различные задачи, а LSTM-модели – мощнейший инструмент для работы с последовательностями.

Стоит отметить NeuralProphet – улучшенную версию классического Prophet. Симбиоз нейросети и статистической модели хорошо справляется с прогнозированием финансовых временных рядов, Prophet-часть модели – распознает линейные и нелинейные тренды через регрессию, а нейросети, учитывая эти знания, выдают итоговый прогноз. Модель уверенно справляется с переобучением при помощи регуляризации в регрессии и Dropout в нейросетях, что дает значительный прирост в метриках.

Подводя итоги, добавлю, что для всех, используемых в данном исследовании моделей, для эффективной работы должно выполняться ряд условий в данных. Если LightGBM и LSTM – универсальные инструменты, для работы с которыми стоит добавить дополнительных признаков, то для использования ARIMA, являющейся статистической моделью, временной ряд должен быть обязательно стационарным. Перед прогнозированием всегда нужно тщательно ознакомиться с данными, проанализировать их, посмотреть на их "природу". На мой взгляд, для подобных задач "симбиозные" инструменты (по типу NeuralProphet)

использовать наиболее практично, так как в случае нестационарности данных или недостатка признаков в них модель сможет грамотно обработать эти проблемы и полученное предсказание в большей части случаев будет адекватным.

Литература

1. <https://bijournal.hse.ru/data/2023/10/02/2062423607/6.pdf>
2. <https://neuralprophet.com/>
3. "Практический анализ временных рядов" - Эйлин Нильсен
Сбор
4. <https://education.yandex.ru/handbook/ml/article/modeli-vida-arima>
5. <https://education.yandex.ru/handbook/ml/article/analitika-vremennyh-ryadov>